

Problème 1 : L'image de base est beaucoup trop grosse

1. Quelle ligne est en cause ?
2. `FROM node:18`
3. Quel est le problème exact ?
4. On utilise la version "complète" de Node.js. Elle embarque tout un système d'exploitation entier avec plein de logiciels qui ne nous servent à rien pour juste faire tourner notre code.
5. Quelle est la conséquence en production ?
6. L'image finale va être énorme (presque 1 Go). Ça va prendre un temps fou à télécharger et à déployer sur les serveurs. En plus, avoir plein de logiciels inutiles, c'est augmenter les chances qu'il y ait une faille de sécurité dedans.

Problème 2 : Les mots de passe laissés en clair

1. Quelle ligne est en cause ?
2. `ENV API_KEY=sk-prod-abc123...` et `ENV DB_PASSWORD=supersecret123`
3. Quel est le problème exact ?
4. Le développeur a écrit les mots de passe de la base de données et de l'API directement dans le fichier.
5. Quelle est la conséquence en production ?
6. C'est une catastrophe niveau sécurité. N'importe qui qui récupère l'image ou qui tape une simple commande (`docker inspect`) pourra lire ces mots de passe en clair et pirater nos bases de données.

Problème 3 : Le cache Docker est complètement cassé

1. Quelle ligne est en cause ?
2. `COPY . .` (juste avant le `RUN npm install`)
3. Quel est le problème exact ?
4. On copie tout notre code source *avant* d'installer les dépendances (`npm install`). Le souci, c'est que si on modifie ne serait-ce qu'une seule ligne de notre code, Docker va croire que tout a changé et va jeter son cache.
5. Quelle est la conséquence en production ?
6. À chaque fois qu'on va vouloir créer une nouvelle version de l'image (build), Docker va retélécharger et réinstaller toutes les dépendances depuis zéro. Ça va rendre les déploiements hyper lents.

Problème 4 : L'installation de la "boîte à outils du hacker"

1. Quelle ligne est en cause ?
2. `RUN apt-get update && apt-get install -y curl wget vim git htop net-tools`
3. Quel est le problème exact ?
4. On installe des outils pour fouiller dans les réseaux et télécharger des fichiers (`curl`, `wget`) dans une image qui est censée juste faire tourner une appli.
5. Quelle est la conséquence en production ?
6. C'est un cadeau pour un pirate. S'il arrive à pirater notre application, il aura déjà tous les outils préinstallés pour télécharger des virus, scanner le reste du réseau de l'entreprise ou voler des données.

Problème 5 : L'application tourne en mode administrateur (Root)

1. Quelle ligne est en cause ?
2. L'absence d'une ligne `USER node` à la fin (avant le `CMD`).
3. Quel est le problème exact ?

4. Comme on n'a rien précisé, Docker va faire tourner notre application avec les droits "root", c'est-à-dire avec tous les pouvoirs sur le conteneur.
5. **Quelle est la conséquence en production ?**
6. Si un attaquant trouve une faille dans notre code, il devient immédiatement administrateur ("root") à l'intérieur du conteneur. Depuis cette position, il lui sera beaucoup plus facile d'essayer de s'échapper du conteneur pour aller attaquer le serveur qui l'héberge.

Screen de test :

docker images tp1:corrige

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
tp1:corrige	5021e1ddd578	190MB	46.4MB	

docker run --rm tp1:corrige whoami

```
lucas@luc:~$ docker run --rm tp1:corrige whoami
node
```

docker inspect tp1:corrige

```
{
  "Architecture": "amd64",
  "Config": {
    "Cmd": [
      "node",
      "dist/index.js"
    ],
    "Entrypoint": [
      "docker-entrypoint.sh"
    ],
    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
      "NODE_VERSION=18.20.8",
      "YARN_VERSION=1.22.22"
    ],
    "ExposedPorts": {
      "3000/tcp": {}
    },
    "User": "node",
    "WorkingDir": "/app"
  },
  "Created": "2026-06-22T09:54:46.670904458+02:00",
  "Descriptor": {
    "digest": "sha256:5021e1ddd5785a6966117b48b1afd5388b80c0d013452531d8deac302f654eb9",
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "size": 2269
  },
  "Id": "sha256:5021e1ddd5785a6966117b48b1afd5388b80c0d013452531d8deac302f654eb9",
}
```

```
docker run -p 3000:3000 tp1:corrige
```



TP1 — Image Docker

Envoyer

Messages (1)

bonjour Docker (22/06/2026 08:00:14)

Container : e2b915915710

.dockerignore

```
.dockerignore
1  # Ignorer les dépendances locales
2  node_modules
3  npm-debug.log
4
5  # Ignorer les dossiers liés à Git
6  .git
7  .gitignore
8
9  # Ignorer les fichiers de variables d'environnement locales (sécurité)
10 .env
11 .env.local
12
13 # Ignorer la documentation et nos captures
14 captures/
15 reponses.md
```

```
docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
ghcr.io/aquasecurity/trivy:latest image \
--severity CRITICAL,HIGH tp1:corrig
```

Node.js (node-pkg)							
Total: 11 (HIGH: 11, CRITICAL: 0)							
Library	Vulnerability	Severity	Status	Installed Version	Fixed Version		
Title							
cross-spawn (package.json)	CVE-2024-21538	HIGH	fixed	7.0.3	7.0.5, 6.0.6	cross-spawn: regular exp	
ression denial of service						https://avd.aquasec.com/	
nvd/cve-2024-21538							
glob (package.json)	CVE-2025-64756			10.4.2	11.1.0, 10.5.0	glob: glob: Command Inje	
ction Vulnerability via Malicious						Filenames	
						https://avd.aquasec.com/	
nvd/cve-2025-64756							
minimatch (package.json)	CVE-2026-26996			9.0.5	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3	minimatch: minimatch: De	
nial of Service via specially						crafted glob patterns	
						https://avd.aquasec.com/	
nvd/cve-2026-26996							
	CVE-2026-27903				10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3	minimatch: minimatch: De	
nial of Service due to unbounded						recursive backtracking v	
ia crafted...						https://avd.aquasec.com/	
nvd/cve-2026-27903							
	CVE-2026-27904				10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4	minimatch: Minimatch: De	
nial of Service via catastrophic						backtracking in glob exp	
ressions						https://avd.aquasec.com/	
nvd/cve-2026-27904							